FAST — NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES
FAST School of Computing · Karachi Campus · Department of Artificial Intelligence & Data Science
AL-2002 Artificial Intelligence (Laboratory) · Fall 2026

PROJECT PROPOSAL

Semester Lab Project · Weightage 1 mark

Group Information

	Name	Roll Number	Section	Role in the project
Leader	Amnn Jindani	24K-2584	BDS-5A	Knowledge Base and Bayesian Belief Updating Developer
Member 2	Farwa Batool Naqvi	24L-2508	BDS-5A	Question Selection, User Interface and Evaluation Developer

Project Title	'Guess Who I Am' (Akinator-Style)
Project Idea	Idea No. __5____ from the Project Ideas list
Category	Part A — Game
GitHub Repository URL	
Date of Submission	22-10-26

1. Project Title and Summary

"Guess Who I Am" is an Akinator-style game in which the player secretly thinks of a well-known Pakistani personality or place, and the computer works out who or what it is by asking yes/no questions. After every answer, the program updates how likely each candidate is and then picks the question that will narrow the field the most. It is meant for anyone curious about Pakistani people, history, and geography, and no prior knowledge is needed to play. A plain program could only follow a fixed script of questions, which fails as soon as the player answers "probably" or gives a mistaken answer. The AI instead reasons under uncertainty, keeps a probability for every candidate and chooses a new best question on every turn.

2. Problem Statement and Motivation

Identifying a hidden person or place from yes/no answers sounds easy, but the space of possible interrogations is enormous. With a knowledge base of 40 attributes (for example, "is alive", "is a politician", "is in Punjab"), the number of ordered ways to choose 20 of them is 40!/20!, about 3 × 10^29, so trying every sequence is hopeless. The information is also uncertain: players answer "probably", misremember facts, or disagree with our data, so a single wrong answer can derail a rigid decision tree. The questions interact too, because a good question depends on every answer already given and on which candidates are still likely. Today, people playing such games simply guess at random or ask questions in a fixed order, which wastes turns and breaks on noisy answers. Solving this is worthwhile because it shows how probability and information theory let a program get the most out of every answer, while making Pakistani culture and geography more interactive to explore.

3. Problem Formulation

3.1 PEAS Description

Component	Description for your project
Performance measure	Success rate (correct guess within 20 questions), average number of questions needed, and robustness when the player gives wrong or uncertain answers.
Environment	The player, who holds a secret character, and a knowledge base of Pakistani personalities and places described by yes/no attributes.
Actuators	Streamlit interface that displays each question, the top candidate guesses with their probabilities, and the final guess.
Sensors	The player's answers (Yes / Probably / Don't know / Probably not / No), entered through buttons.

3.2 Environment and Agent Type

Property	Classification — tick one
Observability	Fully observable ☐ Partially observable ☒
Determinism	Deterministic ☐ Stochastic ☒
Change over time	Static ☒ Dynamic ☐
State and actions	Discrete ☒ Continuous ☐
Agents	Single-agent ☒ Multi-agent / adversarial ☐
Agent type	Simple reflex ☐ Model-based ☐ Goal-based ☐ Utility-based ☒ Learning ☐

3.3 Problem Representation

Element	Your representation
State representation	A probability distribution over all N candidates, plus the questions asked so far and the answers received.
Actions/moves	Ask one unused attribute question, or make a final guess.
Initial state	Uniform prior over all candidates (each 1/N) with no questions asked.
Goal or terminal test	Top candidate's probability reaches 0.85, and the player confirms the guess, or 20 questions have been asked.
Cost, utility, fitness or evaluation	Each question costs 1. Candidate questions are scored by expected information gain, i.e. the expected drop in entropy of the distribution.

4. Proposed AI Approach

#	Technique	Role in the project	Own code or library
1	Bayesian belief updating with soft likelihoods	Updates every candidate's probability after each answer and tolerates "probably" and wrong answers.	Own code
2	Information-gain (entropy) question selection	Chooses the unused question that most reduces uncertainty about the secret.	Own code
3	Pandas / NumPy data handling and Streamlit UI	Loads and filters the knowledge base and runs the game interface.	Library

#	Technique	Role in the project	Own code or library
B	Baseline: random question selection	Asks random unused questions with the same Bayesian update; used to measure what technique 2 adds.	Own code

The problem in Section 3 is partially observable and stochastic, so a probability distribution over candidates is the natural way to store what the agent knows. Bayes' rule updates it after each answer. To cope with unreliable answers, we use soft likelihoods: a "Yes" multiplies a matching candidate by about 0.9 and a non-matching one by 0.1, while "Probably" uses gentler values such as 0.7 and 0.3. Because the agent is utility-based, it simulates each possible answer to every unused question, measures the entropy of the resulting distribution, and asks the question with the highest expected information gain, meaning the one that splits the remaining probability most evenly. We compare this against a baseline that picks random questions, which shows how much the heuristic actually contributes.

4.1 Planned Class Design

```
Candidate: name, kind (person/place), attributes (dict)
KnowledgeBase: load_csv(), candidates, attribute_matrix(), questions()
BeliefState: probabilities, update(question, answer), entropy(), top_k()
QuestionSelector (abstract): select(belief, kb, asked) → question
    InfoGainSelector(QuestionSelector): expected_information_gain()
    RandomSelector(QuestionSelector): seeded random choice (baseline)
GameAgent: has a BeliefState and a QuestionSelector; ask(), receive_answer(), guess(), is_done()
SimulatedPlayer: secret, noise_rate, rng; answer(question)
Evaluator: uses GameAgent and SimulatedPlayer; run_experiments(), compute_metrics(), plot_charts()
app.py: Streamlit UI that wraps GameAgent
```

5. Scope

In scope — the features you commit to delivering. List the features that must work for the project to be considered complete.

#	Feature	Why it is essential
1	Knowledge base of about 150–200 personalities and places with about 40 yes/no attributes	The agent has nothing to reason over without it.
2	Bayesian update supporting five answer options	This is the core AI technique and handles uncertain answers.
3	Information-gain question selection	It makes the agent intelligent rather than a fixed script.
4	Streamlit game interface showing the current top guesses	It lets a real user play the game.
5	Automated evaluation against the random baseline using a simulated player	It provides evidence that the AI works.
6	Ability to add new characters when the AI guesses incorrectly.	It allows the knowledge base to grow and improves the game's ability to identify new characters in future rounds.

Out of scope — deliberately excluded. Naming what you will not build is as important as naming what you will; it shows that your scope is realistic for a group of two.

Free-text or voice input, learning new characters from players during play, a live web scraper, multiplayer or leaderboard features, mobile app packaging, and Urdu-language support.

6. Data or Environment Source

Source name and URL	Hand-curated knowledge_base.csv, compiled from public facts (e.g. Wikipedia and Wikidata, https://www.wikidata.org) and stored in the repository.
Type	Public dataset ☐ Web API ☐ Self-collected ☒ Generated / simulated ☐
Access requirements	No key needed ☒ Free key required ☐ (Paid sources are not permitted)
Approximate size	About 150–200 rows × 40 attribute columns (under 1 MB).
Licence/terms of use	Our own compilation of factual data. Wikidata facts are CC0, and no Wikipedia text is copied.
Offline fallback	The CSV lives in the repository and is loaded locally, so no network is needed at runtime. Any data fetched while building it is saved to a local cache and read from there when offline.

7. Evaluation Plan

What will be compared	Information-gain question selection versus the random-question baseline (optionally also a fixed-order question list).
Metric(s)	Success rate within 20 questions, average questions to a correct guess, and run time per game.
Baseline	Random question selection using the same Bayesian update.
Test instances/test set	Every candidate in the knowledge base used as the secret by a simulated player, at 0%, 5%, and 10% answer noise.
Number of runs and seed	10 runs per candidate per noise level, fixed seed 42 for every approach.
Planned charts (at least three)	(1) Success rate by approach and noise level; (2) average questions to guess versus noise; (3) distribution of questions needed; (4) entropy versus question number.

8. Division of Work

Member	Modules/components owned	Approximate share
Amnn Jindani	Knowledge base, Belief State and Bayesian update, report	50%
Farwa Batool Naqvi	Question selectors, Streamlit UI, SimulatedPlayer, Evaluator and charts	50%

9. Milestones and Timeline

#	Milestone — what will be demonstrably working	Target date	Owner
1	Knowledge base of 100+ entries loads in Pandas and a console game runs with random questions	30 Oct 2026	Amnn
2	Bayesian update works: the console game narrows candidates and prints the top 5 with probabilities	6 Nov 2026	Amnn
3	Information-gain selector beats the random baseline on 20 test secrets	13 Nov 2026	Farwa
4	Streamlit app is playable end to end, with noise handling and the simulated-player evaluator	27 Nov 2026	Farwa

#	Milestone — what will be demonstrably working	Target date	Owner
5	Experiments complete, final integration, documentation and repository cleanup	4 Dec 2026	Both

10. Expected Outcome and Risks

When run, the Streamlit app asks the player to think of a Pakistani personality or place, then asks questions one at a time while showing a live list of the most likely candidates. We expect it to identify the secret in roughly 10–12 questions on average, compared with about 17 or more for random questions, and to keep a high success rate with 5–10% wrong answers. Our report will present the charts from Section 7.

Risk 1: the knowledge base may be too small or have uneven attributes, so some questions split candidates poorly. We would add attributes and candidates and drop questions with near-zero information gain.

Risk 2: “probably” and wrong answers may confuse the model. We would tune the likelihood values and ask a confirming question before making a final guess.

For office use — Lab Instructor	
Proposal status	Approved ☐ Approved with changes ☐ Revision required ☐
Comments	
Signature and date	

End of Project Proposal